

TASK 1 — Create OSSP Project Structure

```
krishnasree08@krishnasree08:~/practical_week2$ mkdir OSSP
mkdir: cannot create directory 'OSSP': File exists
krishnasree08@krishnasree08:~/practical_week2$ cd OSSP
krishnasree08@krishnasree08:~/practical_week2/OSSP$ mkdir src include obj bin docs tests scripts assets
krishnasree08@krishnasree08:~/practical_week2/OSSP$ touch .gitignore README.md LICENSE Makefile
krishnasree08@krishnasree08:~/practical_week2/OSSP$ touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
krishnasree08@krishnasree08:~/practical_week2/OSSP$ touch include/shell.h include/parser.h include/executor.h include/builtin.h
krishnasree08@krishnasree08:~/practical_week2/OSSP$ touch docs/design.md docs/report.pdf
krishnasree08@krishnasree08:~/practical_week2/OSSP$ touch tests/test_parser.c tests/test_executor.c
krishnasree08@krishnasree08:~/practical_week2/OSSP$ touch scripts/run.sh assets/architecture.png
krishnasree08@krishnasree08:~/practical_week2/OSSP$ find . -maxdepth 3 -print
.
./docs
./docs/report.pdf
./docs/design.md
./tests
./tests/test_executor.c
./tests/test_parser.c
./Makefile
./scripts
./scripts/run.sh
./include
./include/executor.h
./include/builtin.h
./include/parser.h
./include/shell.h
./src
./src/builtin.c
./src/main.c
./src/parser.c
./src/executor.c
./src/shell.c
./obj
./README.md
./bin
./assets
./assets/architecture.png
./.gitignore
./LICENSE
```

Summary:

The required OSSP project structure was created in Ubuntu using `mkdir` and `touch` commands. The structure includes directories such as `src`, `include`, `obj`, `bin`, `docs`, `tests`, `scripts`, and `assets`, along with the required source, header, documentation, test, and configuration files

Result

Thus, the required OSSP project structure was successfully created in Ubuntu.

TASK 2 — Understand `fork()` and `exec()`

```
krishnasree08@krishnasree08:~/practical_week2/OSSP$ man 2 fork
krishnasree08@krishnasree08:~/practical_week2/OSSP$ q
Command 'q' not found, but can be installed with:
sudo snap install q # version 1.6.3-1, or
sudo apt install python3-q-text-as-data # version 3.1.6-3
See 'snap info q' for additional versions.
krishnasree08@krishnasree08:~/practical_week2/OSSP$ man 3 exec
```

Summary:

The `fork()` and `exec()` system calls were studied using Linux manual pages. `fork()` is used for creating a child process, while the `exec()` family is used to execute another program.

Result:

Thus, the `fork()` and `exec()` system calls were successfully studied in Ubuntu.

TASK 3 — C Program Using `fork()`

```

krishnasree08@krishnasree08:~/practical_week2/OSSP$ cd src
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ nano process.c
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ gcc process.c -o
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ ./process
This is the Parent Process.
Parent PID = 665
Child PID = 666
This is the Child Process.
Child PID = 666

```

Summary:

A C program was created using the fork() system call. The program successfully creates a child process and displays the Process IDs of the parent and child processes.

Result:

Thus, a new process was successfully created using the fork() system call.

TASK 4 — Accept User Command Using fork() and exec()

```

krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ ~/OSSP/src
-bash: /home/krishnasree08/OSSP/src: Is a directory
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ nano command_exe
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ gcc command_exec
krishnasree08@krishnasree08:~/practical_week2/OSSP/src$ ./command_execut
Enter a command: ls
This is the Parent Process.
Parent PID = 688
Child PID = 689
This is the Child Process.
Child PID = 689
builtin.c  command_executor  command_executor.c  executor.c  main.c  par
Child process completed.

```

Summary

A C program was successfully implemented using fork() and exec(). The program accepts a Linux command from the user, creates a child process, executes the command through the child process, and waits for the child process to complete.

Result

Thus, the user-specified Linux command was successfully executed using the fork() and exec() system calls.